

STRUKTURALNI PATERNI

U dokumentu su navedeni strukturalni paterni koji bi se mogli primijeniti u FitManager sistemu. U klasnom dijagramu implementirana su samo dva odabrana paterna: Facade i Proxy.

ADAPTER PATERN:

- Adapter patern služi za prilagođavanje interfejsa postojeće klase interfejsu koji klijent očekuje.
- U FitManager sistemu ovaj patern bi se mogao primijeniti na sistem obavještenja, npr. ako se EmailObavjestenje kasnije povezuje sa SMS, push ili eksternim mail servisom.
- Ovaj patern bi riješio problem različitih načina slanja obavještenja bez mijenjanja klase koja pokreće obavijest.

BRIDGE PATERN:

- Bridge patern služi za razdvajanje apstrakcije od implementacije kako bi se obje mogle nezavisno mijenjati.
- U FitManager sistemu ovaj patern bi se mogao primijeniti na obavještenja, tako da se tip obavještenja odvoji od načina slanja.
- Na primjer, potvrda rezervacije, podsjetnik za trening i obavijest o isteku članarine mogli bi se kombinovati sa email, SMS ili push slanjem.

COMPOSITE PATERN:

- Composite patern služi za organizovanje objekata u strukturu stabla, tako da se pojedinačni objekti i grupe objekata mogu tretirati na isti način.
- U FitManager sistemu ovaj patern bi se mogao primijeniti na izvještaje, gdje bi pojedinačni izvještaj i grupa izvještaja imali zajednički interfejs.
- Na taj način bi administrator mogao raditi sa dnevnim, sedmičnim ili mjesečnim izvještajima bez posebnog razlikovanja da li je u pitanju jedan izvještaj ili kolekcija izvještaja.

FACADE PATERN:

- Facade patern služi za pojednostavljenje korištenja složenog podsistema preko jedne jasne ulazne tačke.
- U FitManager dijagramu ovaj patern je primijenjen na proces rezervacije grupnog treninga kroz klasu RezervacijaTreningaFacade.
- Ovaj patern rješava problem direktne komunikacije kontrolera sa više klasa: Clanarina, GrupniTrening, Rezervacija, EmailObavjestenje i ApplicationDbContext.
- RezervacijaTreningaFacade nudi metodu RezervisiTrening(clanId, treningId), a u pozadini koordinira provjeru kapaciteta, provjeru postojeće rezervacije, kreiranje rezervacije, umanje broja slobodnih mjesta i evidentiranje obavještenja.

PROXY PATERN:

- Proxy patern služi kao zastupnik stvarnog objekta i kontroliše pristup prije nego što zahtjev dođe do glavne poslovne logike.
- U FitManager dijagramu ovaj patern je primijenjen kroz klasu SigurnosniRezervacijaProxy.

- Proxy implementira isti interfejs IRezervacijaProces kao i stvarni proces rezervacije, pa RezervacijeController ne mora znati da li komunicira sa proxy klasom ili fasadom.
- Prije prosljeđivanja zahtjeva, SigurnosniRezervacijaProxy provjerava da li član postoji i da li ima aktivnu članarinu. Ako uslov nije ispunjen, vraća RezultatRezervacije sa porukom neuspjeha i zaustavlja tok.

DECORATOR PATERN:

- Decorator patern služi za dinamičko dodavanje novih odgovornosti objektu bez mijenjanja njegove osnovne klase.
- U FitManager sistemu ovaj patern bi se mogao primijeniti na obračun cijene članarine ili pogodnosti, npr. sezonski popust, studentski popust ili promo kod.
- Ovaj patern bi riješio problem krutog nasljeđivanja, jer bi se različiti popusti mogli kombinovati bez prepravljjanja klase Clanarina ili TipClanarine.